

[Iron-Clad Java](#), [Jim Manico](#), [August Detlefsen](#)

Published by [McGraw Hill Computing](#)



CHAPTER 7

SQL Injection and Other Injection Attacks

Make no mistake: Many web and web service applications are simply glorified database applications. Programmers interface their web applications to databases via structured query languages such as SQL, OQL, and HQL. Improperly structured queries are a leading cause of exploitation in online applications and are infamously known as *SQL injection*!

What Is SQL Injection?

SQL injection occurs when *untrusted data* such as user data from application web pages are added to database queries, materially changing the *structure* and producing behaviors inconsistent with application design or

purpose. Clever attackers exploit SQL injection vulnerabilities to steal sensitive information, bypass authentication or gain elevated privileges, add or delete rows in the database, deny services, and in extreme cases, gain direct operating system shell access, using the database as a launch point for sophisticated attacks against internal systems.

SQL injection is by far the most dangerous vulnerability impacting online applications today. When you read that hackers have stolen 450,000 user accounts or that 130 million credit card numbers have been exposed it is very likely that the application was attacked using SQL injection. Fortunately, there are many tools and techniques you can leverage to protect your applications. These software tools generally fall into two categories: *dynamic analysis* and *static analysis*. Dynamic analysis tools work by scanning applications at runtime to test for SQL injection by rapidly testing a large number of known attack patterns and looking for variations in the server's response, which indicates the presence of vulnerability. Once a vulnerability is confirmed, these tools can automate query construction to facilitate easy review of database structure or modify data. Static analysis tools fulfill the same purpose but instead test program *code*. Static analysis tools are beneficial because code can be reviewed or analyzed as it is developed. Any vulnerability discovered can be remediated earlier in the development process, reducing overall costs of the project. Dynamic analysis tools are beneficial when enough work has been completed so the application can execute. The best practice is to combine both techniques as you include security testing into your development processes.

What does SQL injection look like in code? The main security anti-pattern leading to SQL injection is use of “dynamic string building” techniques to assemble a database query in code at runtime. Specifically, when programmers mix query language fragments and untrusted data, it creates an opportunity for SQL injection.

Let's start with an example in Java code.

```
 // CAUTION: Please do not use this code sample.  
// CAUTION: It's dangerous and vulnerable to SQL injection!  
String sqlQuery = "SELECT id, name, short_name, size_code " +  
    " FROM company " +  
    " WHERE id = " + request.getParameter("company_id");
```

The first part of the structured query fragment is `SELECT id, name, short_name, size_code FROM company WHERE id =`. Next is raw untrusted data—in this case, user

input from a web application, resolved, and concatenated to the previous string: `request.getParameter("company_id")`.

The query fragments are assembled into a single query and sent to the database for processing.



TIP

The value of `request.getParameter()` can only be determined at runtime and is the injection vector. Building the database query string dynamically in combination with untrusted data creates an opportunity for the database to be compromised.

How dangerous is SQL injection? There are many ways attackers can manipulate part of the HTTP request to exploit an SQL injection vulnerability and do harm. For example, what would happen in the preceding example if the user submitted `1 or 1=1` as the `company_id` parameter? Let's take a close look. On the web browser, the URL sent to the application may look something like:

 `http://www.vulnerable.com/company?company_id=1+or+1%3d1`

When the previous input is concatenated together with our original SQL statement it produces the following structured query to be executed by the database (tainted input shown in **bold**):

 `select id, name, short_name, size_code from company
where id=1 or 1=1`

With the previous query, it should not be hard to guess what will happen next. The "`or 1=1`" added to the `WHERE` clause results in a condition that always returns true! Instead of returning a single company, the SQL statement now returns every company record in the database.

The preceding is a relatively tame example. For something more compelling, consider the following example of a forms-based authentication mechanism. An SQL query used to authenticate an online web application user might look something like the following:

```
// CAUTION: Please do not use this code sample.
// CAUTION: It's dangerous and vulnerable to SQL injection!

"SELECT id, first_name, last_name, permissions " +
    " FROM user " +
    " WHERE username = '" + request.getParameter("username") +
    "' AND password = '" + request.getParameter("passwd") + "'";
```

Consider what would happen if the following malicious HTTP POST is received by the application server:

```
POST /login HTTP/1.1
Host: www.vulnerable.com

username=fred&password=a'+or+'a'%3d'a
```

The resolved structured query executed by the database will look like this:

```
SELECT id, first_name, last_name, permissions
FROM user
WHERE username = 'fred'
AND password = 'a' or 'a'='a'
```

Any idea how this would impact the application logon? The result is a complete authentication bypass for the user `fred` by making his password irrelevant. By changing the `username` parameter, an attacker can log in as any user on the system and gain full privileges to all services and data available to that user.

What if that password is hashed? (Please note that proper password storage techniques are discussed in [Chapter 3](#).) In that case, your code might look like this:

```
// CAUTION: Please do not use this code sample.
// CAUTION: It's a VERY WEAK way to store a password!
String hashedPassword = sha256(request.getParameter("password"));

"SELECT id, first_name, last_name, permissions " +
    " FROM user " +
    " WHERE username = '" + request.getParameter("username") +
    "' and password = '" + hashedPassword + "'";
```

An unsalted SHA-256 hash of the same attack string used in the previous example might look like this:

 075f6762743a6e4ad5d3af93f4c3d805f669b2cacedeafc71c02649e5dcc112d

making the ultimate query executed by the database look like this:

```
 SELECT id, first_name, last_name, permissions
FROM user
WHERE username = 'fred'
AND password =
'075f6762743a6e4ad5d3af93f4c3d805f669b2cacedeafc71c02649e5dcc112d'
```

So that is safe, right? By hashing the password, an attacker can no longer alter the structure of the SQL query, right? Wrong.

The `username` parameter is still directly concatenated, which makes the entire SQL statement vulnerable. Remember that if an attacker can insert SQL fragments anywhere into the query then the entire SQL statement is vulnerable. For instance, if an attacker sends the input `fred'--` (`--` is a comment in SQL), the resulting statement executed is:

```
 SELECT id, first_name, last_name, permissions
FROM user
WHERE username = 'fred'--'
AND password =
'075f6762743a6e4ad5d3af93f4c3d805f669b2cacedeafc71c02649e5dcc112d'
```

Again, this would result in a complete bypass of the password-based authentication mechanism because the remainder of the query following `fred'--` is considered a comment and not part of the query considered for execution. This also brings up an interesting point: Different databases have slightly different grammar and capabilities. Some injection attempts may only be successful against specific databases or even versions.

Other SQL Injection Examples

A variety of automated tools exist to facilitate data exfiltration of an entire database one character at a time by attackers. Even the most “minor” SQL injection vulnerabilities can lead to complete database theft or compromise.

Okay, you get it—SQL injection is dangerous! So how do you stop it? Following are a few suggestions to prevent or stop SQL injection attacks that are commonly cited but are *not proper defenses* against SQL injection! *Do not depend on these to stop SQL injection!*

■ **Blacklist validation** This defense technique sets up rules to define SQL injection attacks and then blocks that request. This is a very challenging and often fragile defense. If the blacklist validation rule is too strong, then good

requests get blocked. If the blacklist validation rule is too loose, then real-world attacks get through.

■ **Output encoding** This defense technique “escapes” input before dynamically adding that input to a dynamic query. Different kinds of encoding (done by the attacker on input) can bypass this defense and escape characters needed for proper SQL encoding can be changed on the database (and will be different for each database vendor).

■ **Stored procedures** As we will demonstrate shortly, stored procedures alone are not enough to stop SQL injection. In fact, if built incorrectly, stored procedures themselves can be vulnerable to SQL injection.

The previously described methods are either too strict or too loose or can cause security vulnerabilities in and of themselves.¹ Blacklist filters are often put in place as temporary solutions where a more permanent protection would be more appropriate. Admittedly, blacklist filtering *in addition* to whitelist filtering can provide good intrusion detection.

You cannot simply blacklist the SQL quote character (') because in some cases you may need to accept that as valid input. Consider the last name “O’Leary,” for example, or perhaps a user’s name is actually “NULL.” These characters need to be stored unchanged in the database to be represented correctly when they are rendered in the user’s browser.

There are multiple defense methods to consider when designing secure web and web service applications to be resistant to SQL injection. The two primary methods are *query parameterization* and *input validation*. There are also mechanisms that can be put in place to reduce the impact of SQL injection in a vulnerable application. Some of these suggestions are just good design and good software security hygiene.

It is preferred to not alter untrusted input data directly but instead to prevent it from changing the behavior of the structured query as originally designed. In other words, even if the untrusted data contains reserved characters or words used by the database, they will be interpreted by the database as data and not as commands. This leads us into our next topic: parameterization. Let’s dig in!

Query Parameterization

Fortunately, Java already includes a built-in mechanism to prevent SQL injection: prepared statements. Prepared statements are Java's flavor of *query parameterization*, by far the most important software engineering technique in the battle to stop SQL injection. Query parameterization is a computer programming technique to build a *dynamic SQL statement* safely by "binding" untrusted data into placeholders within a query.

Because of the level of the risk posed by SQL injection, query parameterization is *the most important* technique a developer can use to build a secure website. Proper use of query parameterization to build dynamic database queries will protect you from SQL injection attacks.

Most developers are likely familiar with the use of prepared statements in Java because of the performance improvement they provide. The other benefit of prepared statements is that they effectively separate query *data* from query *structure* in SQL queries. Why do parameterized queries run faster? The query plan for parameterized queries is pre-generated in the database. Subsequent calls to the same query require little additional processing so queries run on the same connection will perform faster. So not only is this mechanism more secure, it also forces the developers to build better designed code, which is easier to maintain over time.

Following is our first example of "injection safe" database queries with query parameterization. In Java, query parameterization is provided via the `java.sql.PreparedStatement` class.²

```
String custname = request.getParameter("customerName");
String query =
    "SELECT profile_desc FROM users WHERE user_name = ?";
PreparedStatement pstmt = connection.prepareStatement( query );
pstmt.setString( 1, custname);
ResultSet results = pstmt.executeQuery( );
```

When you utilize the programming technique of query parameterization, it's actually the *database* that protects your application from injection. When you use the `PreparedStatement` class and similar constructs, the parameterized SQL (the portion of your query with your placeholder variables, not the untrusted data) is sent to the database, which causes the database to build a query execution plan for that query structure. This query execution plan is cached at the database level. The second part of your prepared statements, the binding of the untrusted data into the query, is then sent to the database in a second pass, to traverse the query plan and execute the query. Because the plan is "built" already, injection is impossible. Luckily, query parameterization in Java is one of the easier secure coding techniques that you need to master.

Also, not only does query parameterization protect against SQL injection, but it also increases the performance of repeated queries because your query execution plan is cached in the database! This makes query parameterization a security control with multiple benefits beyond security.

```
String untrustedEmployeeName=request.getParameter("user_name");
String sessionId=session.getAttribute("user_id");
if ((untrustedEmployeeName != null) && (sessionId != null)) {
    PreparedStatement pstmt = con.prepareStatement(
        "UPDATE EMPLOYEES SET NAME = ? WHERE ID = ?");
    pstmt.setString(1, untrustedEmployeeName);
    pstmt.setString(2, sessionId);
}
```

In the `UPDATE` statement example in the preceding code, even though the `userId` is bound to the query as a string, the query is still fully immune to SQL injection! Also, even though the `userId` comes from a trusted source (the session), you still parameterize that variable into the query for completeness, performance, and auditability.

The best defense against SQL injection is query parameterization for all queries and the binding of all variables that need to be dynamically added to each query. When it comes to your application, *don't jeopardize, just parameterize!*

SQL Injection and Stored Procedures

Even stored procedures can be “injectable.” Database designers and developers must also use query parameterization and variable binding when developing stored procedures as well. This is no fault of the database vendor, and stored procedures can be abused or used insecurely. Fortunately, the technique of query parameterization is easy, more secure, and improves the performance of stored procedures because parameterized SQL is pre-compiled at the database.

The following is an example of a PL/SQL stored procedure that is vulnerable to SQL injection:


```

CREATE FUNCTION user_count_last_name (
    input_last_name IN VARCHAR2
) RETURN int AS
    total_users int;
    final_query constant varchar2(32767) :=
        'SELECT count(*) FROM users
        WHERE lastname= ''' || input_last_name || ''';
BEGIN
    EXECUTE IMMEDIATE final_query INTO total_users;
    RETURN total_users;
END;

```

Can you see the problem in the preceding stored procedure? The variable `input_last_name` is being dynamically added to the query with string building. The attacker can send in a query fragment such as `' or 1=1--` and exfiltrate the total number of users.

The count of users in the database is not that interesting in itself, but it does provide an attacker with a simple indicator for finding other more interesting or useful data. Sending an attack string like the following:

```

'xxxx' OR CASE WHEN SUBSTR(password, 1, 1) = 'a' AND ROWNUM = 1 --

```

would cause the query executed to become:

```

SELECT count(*)
FROM users
WHERE lastname= 'xxxx'
OR CASE WHEN SUBSTR(password, 1, 1) = 'a' AND ROWNUM = 1 --

```

It is not very likely that there is a `lastname` like `'xxxx'` in the database so that part of the `WHERE` clause will evaluate to `FALSE`. But if the first character of the first user's (hopefully hashed) password is `a`, this statement will return 1. If it returns 0, then the attacker knows the first character is not `a`. By rapidly iterating through the alphabet at each position in the password column, an automated tool such as SQLMap³ can retrieve the contents of an entire database, one character at a time!

The solution for securing the previously described stored procedure so that it is immune to SQL injection is straightforward. The developer simply needs to replace the concatenated SQL statement with the version of query parameterization that is available in the database's own stored procedure language (highlighted in **bold**):

```

CREATE FUNCTION user_count_last_name (
    input_last_name IN VARCHAR2
) RETURN int AS
    total_users int;
    final_query constant varchar2(32767) :=
        'SELECT count(*) FROM users WHERE lastname= :input';
BEGIN
    EXECUTE IMMEDIATE final_query INTO total_users USING input_last_name;
    RETURN total_users;
END;

```

As you can see, building injection-resistant stored procedures is not challenging. In fact, it's one of the easiest secure coding techniques to master.

Defense in Depth

Query parameterization is the most powerful technique available, in any web language, to completely stop SQL injection—*forever*. So is any other defense necessary? Other defenses such as input validation, properly designed stored procedures, and limiting the privileges of the user account your application uses to connect to the database are all good mechanisms to lower the overall risk to your application.

There are situations where query parameterization is not possible in code. These exceptions are rare but do come up in legacy code. For example, table names and column names are not parameterizable for some database vendors. Although this situation leads one to ask, “Why is untrusted data driving a column name or table name?” they are often used by developers to create reports that include only certain columns or allow end users to specify custom sort options. This anti-pattern is called *Insecure Direct Object Reference*⁴ and it is the OWASP Top Ten 2013 risk #4. When it is necessary to rapidly fix these types of vulnerabilities in legacy code, and query parameterization is not possible, techniques such as input validation are acceptable alternatives.

Following is an example of injectable code allowing an attacker to specify the column name in a query. In addition to injection, the example exposes the inner structure of your database and application. Suppose we had a URL like this:

```

http://www.site.com/productReport?orderby=price

```

The URL parameter `orderby` corresponds to a database column name dynamically added to an SQL statement and used to order data. The very vulnerable code may look like this:

```
String orderBy = request.getParameter("orderBy");
String dangerousQuery =
    " SELECT name, type, size, num_sold, price " +
    " FROM products ORDER BY " + orderBy;
```

Keep in mind that the parameter name `orderBy` makes it easy for the attacker to guess how the submitted information is used in the application query. Because query parameterization is not possible on column names, the developer must instead carefully validate the `orderBy` input against a list of acceptable values:

```
private static final HashMap<String,String> sortClauses =
    new HashMap<String,String>();
static {
    sortClauses.put("default", "name ASC");
    sortClauses.put("n", "name ASC");
    sortClauses.put("t", "type ASC");
    sortClauses.put("s", "size ASC");
    sortClauses.put("num", "num_sold DESC");
    sortClauses.put("p", "price DESC");
}

String sortId= request.getParameter("sortId");
String orderByClause = sortClauses.get( sortId );
if (Utils.isEmpty(orderByClause)) orderByClause =
    sortClauses.get("default");

String saferQuery= "SELECT name, type, size, num_sold, price " +
    " FROM products ORDER BY " + orderByClause;
```

Indeed, there are many defensive measures protecting your web application from SQL injection. However, it is essential to remember that the key technique every developer needs to know is query parameterization. That's because when you use it to build queries on a website you will *never* have an SQL injection attack on that site. Furthermore, query parameterization is very simple to perform and is a necessity for securing against SQL injection.

Input Validation and Type Safety

Here are a few additional defenses to consider when building safe and effective database interaction.

■ **Verify the number of results** It is often prudent to verify that the number of returned results is within a legal range. If you are expecting one or zero results but receive more than one, the request may very well include an attack or at least some kind of design flaw.

■ **Type safely** If you are pulling out numbers or dates from a `SELECT` statement, force that output into a formal data type. This can help protect against tools that use multiple queries to exfiltrate data one character at a time.

■ **Parameterize every variable** Ensure that even non-injectable variables such as numerics are parameterized for performance and auditability (all variables should be parameterized in SQL code, check!).

DAO Pattern and Access Control Considerations

One of the most common object-oriented design patterns is called the DAO (Data Access Object) pattern. The DAO pattern is useful for the representation and management of application data as application objects. The DAO pattern typically provides the Create, Read, Update, and Delete (CRUD) methods, thus insulating applications from the details required to retrieve the database information. When building using a DAO layer, it is important to ensure that it uses parameterization techniques to keep the application completely injection safe. An easy way to determine if a DAO class is injectable is to look for any DAO API accepting unparameterized query fragments such as table names or column names as parameters. If so, your API is potentially vulnerable to SQL injection. Security concerns aside, passing query fragments to DAO APIs violates the intended purpose of the DAO pattern, which is database neutrality. Other considerations when designing DAO APIs include integrating some minimal input validation to ensure that data is consistent with business requirements and adding identity and access control at the DAO layer. This will ensure that even novice programmers using your DAO API will do so safely.

For example, many DAO layers have an interface that is similar to the following simplistic example:

```
public interface UserDao {  
    public List<User> getAllUsers();  
    public User getUser(int userNum);  
    public void updateUser(User user);  
    public void deleteUser(User user);  
}
```

Please note that this DAO example must include parameterization of all untrusted input! The `userNum`, as well as any data used in the `getUser` and `updateUser` functions, must be parameterized so that no matter what input is sent into this DAO layer, the underlying queries are fully

immune to SQL injection. But please note that an entity that has access to this DAO layer can retrieve all users, update any user, and delete any user. We suggest adding security controls directly to the DAO layer instead of providing that defense “upstream.” A “secured” DAO layer may look like the following:

```
public interface UserDao {
    public List<User> getAllUsers(Entity requestor)
        throws AccessControlException, AuthenticationException;
    public User getUser(Entity requestor, int userNum)
        throws AccessControlException, AuthenticationException;
    public void updateUser(Entity requestor, User user)
        throws AccessControlException, AuthenticationException,
            ValidationException;
    public void deleteUser(Entity requestor, User user)
        throws AccessControlException, AuthenticationException,
            ValidationException;
}
```

SQL Injection and Object Relational Mapping

Even object relational mapping mechanisms such as Hibernate, Castor, or Java’s native JDO are injectable. Consider the following simple Hibernate example showing an object query that is injectable:

```
String productName = request.getParameter("product_name");

Query hibernateQuery = session.createQuery(
    "from Products as products where product.name = '" +
    productName + "'");
List queryResults = hqlQuery.list();
```

Make no mistake, HQL injection is often much more difficult to exploit than normal SQL injection, but the risk is real. Here is an example of an HQL query that is safe from injection:

```
String productName = request.getParameter("product_name");

Query hibernateQuery = session.createQuery(
    "from Products as products where product.name = :prodname");
List queryResults =
    hqlQuery.setString("prodname", productName).list();
```

Reducing the Impact of SQL Injection

There are several configuration measures that can be taken to reduce the impact of SQL injection. If your application is built entirely with parameterized queries, your risk of SQL injection is removed. However, a multilayered defense strategy is crucial when it comes to the construction of

secure applications. The defenses described in the sections that follow will ensure that even if SQL injection somehow slips through your review processes, the damage done to your organization will be contained.

Database Permissions

Imagine a Java-based marketing web application that is backed by a database-driven content management system. The data within this web application may be public data that is not considered sensitive to the business. An SQL injection vulnerability in such an application may be deemed to be low impact. Indeed, security is often overlooked for such sites.

However, sometimes it is difficult for IT operations staff to understand all the database permissions and requirements for every application. To reduce operational impacts of a permission failure they sometimes take shortcuts such as using the database Administrator account as the application's account. Because the Administrator account has privilege to the entire database, the application is not likely to encounter surprise problems with permissions at runtime. The problem with such shortcuts is that Administrative accounts may have access to database schemas of other applications than the marketing web application. Unfortunately, providing an account with too much privilege is not typically discovered during functional quality testing. However, in such a scenario, attackers could use an SQL injection vulnerability to extract data from different database schemas and different applications. In some cases, an attacker can use SQL injection to open an operating system command shell and execute operating system commands on the database server!⁵

The solution is to configure your database connection pool user or standard database user to follow the POLP or POLA principle: *principle of least privilege* or *principle of least authority*. Applying the principle to the previous example, operations staff should have confirmed the proper access requirements with the developers and created a new database user with the *least access possible* to still achieve the desired functionality of the application. For example, the web application might only need to `SELECT` data from certain tables and not actually need the ability to `UPDATE` anything. The web application almost certainly does not need the ability to `CREATE` OR `DROP` tables, create stored procedures, or install additional procedural languages, especially for databases or schemas outside the application itself.

Verification of Number of Results

Verifying the number of results of a database query can help to protect your application. If you are expecting a query to return only one result and it returns multiple results, this might indicate an attack against your application. As a defense-in-depth mechanism, consider checking the number of rows returned or affected by a query and either returning an error message or rolling back the transaction if unexpected results occur.

The `JDBC4ResultSet` class from Java 6 and greater has a method called `getUpdateCount` that returns the number of the lines affected by an SQL statement, making this check easy. This method works for `SELECT` as well as `UPDATE` or `INSERT` statements. The following example shows how to use this technique:

```
 PreparedStatement ps = con.prepareStatement(  
    "SELECT * FROM users WHERE username=? AND passwordhash=?");  
pstmt.setString( 1, usernameInput);  
pstmt.setString( 1, hashedPassword);  
JDBC4ResultSet rs = (JDBC4ResultSet)ps.executeQuery();  
int rowNumber = rs.getUpdateCount(); //works for select as well  
if (rowNumber > 1) {  
    throw SecurityException();  
}
```

With all of these ideas in mind, starting with query parameterization, you should have a complete and robust strategy to never have to deal with SQL injection damaging your applications ever again. Instead of “game over,” proper design and secure coding will allow you to truly be agile when building software in today’s dangerous threatscape. Game on!

Other Forms of Injection

There are many forms of injection beyond SQL injection. All forms of injection are born from the same anti-pattern of mixing untrusted data and string-based commands. As the 90s wonderband The Offspring said in their song “Come Out and Play,” “You gotta keep ’em separated.” These folks were obviously singing about secure software and how to defend against injection attacks. Who would have thought we would find such wisdom in a 90s rock band?

XML and JSON-Based Injection

Frequently, XML/JSON parsing occurs at multiple tiers within a web or web service application. A variety of technologies may consume or deliver XML/JSON. And it’s possible that both delivered and received XML/JSON may in part contain untrusted data. This can lead to server-side XML injection, or browser-based cross-site scripting from untrusted JSON.

XML/JSON needs to be sanitized or parsed with safe parsers in a variety of locations within a web application:

1. When the browser is parsing untrusted JSON, a safe parser should be used in JavaScript, such as `JSON.parse`.
2. When web services receive untrusted XML, an XML schema should be applied to ensure proper XML structure. The XML Schema Definition (XSD) allows you to specify a very granular definition of what elements, attributes, and data types can be contained within an XML document.
3. When untrusted XML/JSON is submitted to a user or another service (untrusted XML/JSON leaves a web service in some way), that output needs to be in the proper format. Validation, encoding, and outbound sanitization are necessary.
4. When XML query language (XPath, XQuery) requests are intermixed with untrusted data, query parameterization or encoding is necessary.

The same moral of secure coding holds true. Any time you are assembling some kind of text-based command (OS, SQL, XML, and so on) that is mixed with user or untrusted data, a similar special defense is needed to avoid injection or similar attacks.

Command Injection

On occasion, Java-based web and web service applications need to directly leverage commands on the server's operating system from application code. Although this can be a very dangerous endeavor, it does occur, and direct operating system calls need to be made safe to accomplish important business tasks.

For example, we were once tasked with building a high-performance image manipulation web application. This application not only needed to achieve great scale, but also had to be able to manipulate large images in a very short amount of time. Even on incredibly powerful hardware, Java's 2D imaging API was rather slow in processing even modestly sized images. Our answer was to call the open source ImageMagick image manipulation library directly from the operating system.

```


// This is without a doubt, a very dangerous function from a
// security perspective. Tries to exec the OS level ImageMagick
// command, and then waits for it to finish.
// Logs errors if exit status is nonzero
// Returns true if exit status is 0 (success).
private final boolean execImageMagickCommand(String[] command) {
    log.debug(command[0] + " called via execImageMagickCommand");
    Process proc;
    try {
        log.debug("Trying to execute command " +
            Arrays.asList(command));
        proc = Runtime.getRuntime().exec(command);
    } catch (IOException e) {
        log.debug("IOException while executing " + command);
        return false;
    }
    //timing code removed for brevity
    return (exitStatus == 0);
}

```

The function that calls this constructs the ImageMagick command by (you guessed it) concatenation:


```


// Uses a Runtime.exec() to use imagemagick to perform the given
// conversion operation. Returns true on success, false on
// failure. Does not check if either file exists.
// @param fileIn source image filepath
// @param fileOut converted image
// @param width image width
// @param height image height
private boolean convert(
    String fileIn, String fileOut, int width, int height) {
    String subFrame = "";
    String extension = fileIn.substring(fileIn.lastIndexOf("."));
    // ai and gif both only need look at first layer
    if (".ai".equalsIgnoreCase(extension) ||
        ".gif".equalsIgnoreCase(extension)) {
        subFrame = "[0]";
    }
    // if we are using subframes format like: filename[0]
    if (!"".equals(subFrame)) {
        fileIn = fileIn + subFrame;
    }

    boolean isPDF = false;
    if (".pdf".equalsIgnoreCase(extension)) {
        isPDF = true;
    }

    // build the actual system command
    ArrayList command = new ArrayList(9);

    command.add(CONVERT_PROG);
    if (!isPDF) command.add(fileIn);
        command.add("-flatten");
        command.add("-colorspace");
        command.add("rgb");
        command.add("-resize");
        command.add(width + "x" + height + ">");
    if (isPDF) command.add(fileIn);
    command.add(fileOut);

    // execute the ImageMagick system command
    String[] finalCommand =
        (String[]) command.toArray(new String[1]);
    return execImageMagickCommand(finalCommand);
}

```

If any of the parameters to the `convert` method had been tainted, this could have resulted in rogue system commands being executed directly on the operating system. Our solution at the time was to apply rigorous input validation to the inputs, but it is amazing that nothing ever slipped past us. This may be a testament to the fact that our system was not widely used and

did not become a huge hit like Flickr, despite having similar functionality. Regardless, we were lucky, not smart, in this case.

Dangerous Characters in Input

Any type of string-based command interpreter can be injected upon. LDAP, operating systems, file I/O HTML, XML or JSON parsers, database queries, and more can all be injected upon when trusted data is mixed with untrusted data. It is unfortunately very common for developers to face forms of injection that do not have good APIs to help defend against. In these situations, the developer must understand the specifications of the software she plans to use and escape untrusted input for any characters reserved by the specification. So, for example, in LDAP, characters such as `*`, `<`, `>`, `/`, and so on are all considered “code” and not “user data” when interpreted by the LDAP parser. Proper escaping and input validation is essential to ensure the system does not get untrusted user data confused with reserved characters. For command injection, proper defense depends upon the operating system. However, characters such as `CRLF`, `CR`, `LF`, `null`, `;`, and so on can cause harm in the context of command injection, as previously discussed. Another example is the Java null byte injection vulnerability. Java had a nasty bug for a while that enabled the user to pass in a `<null>` byte to terminate a string, which produced interesting results.⁶ The following null byte attack is used to bypass protection that limits user input to HTML files by checking that the file extension matches “.html.” However, the null byte injection attack that follows allows the attacker to steal other sensitive data.

```
 /* Imagine you are a web application and you are receiving this
 * input from your web server, ouch. If you check the file name
 * ends in .html you might make the assumption it's an html
 * document being accessed. Instead, this will dump the
 * /etc/passwd file!
 */
```

```
String txt = Start.readFile("/etc/passwd\0.html");
```

This vulnerability has been fixed for some time, and of course you keep your JVMs updated for all of your Java applications, right?

Remediation Checklist

To prevent SQL injection, use the following techniques:

- Use query parameterization for all queries and all variables (*most important*).

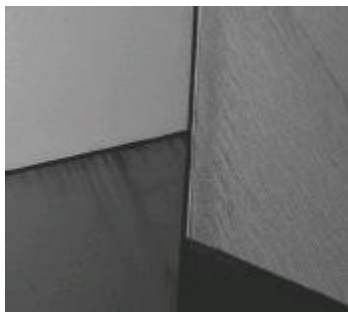
- When query parameterization is not possible, employ whitelist input validation.

- Limit database connection access (Principle of Least Privilege).

- Manually escape all inputs (*last resort*).

Summary

As you can see, injection is one of the most dangerous risks that your applications must contend with. Any time you are building a string-based command that includes untrusted input, injection is possible. When considering the various types of injection, SQL injection is perhaps the most common and dangerous. The varied techniques of input validation, query parameterization, limiting database permissions, and, as a last resort, escaping individual variables will lead you toward building injection-resistant applications.



¹ <http://lists.grok.org.uk/pipermail/full-disclosure/2011-August/082164.html>

² Query parameterization is available in almost every web-centric programming language. https://www.owasp.org/index.php/Query_Parameterization_Cheat_Sheet

³ <http://sqlmap.org/>

⁴ https://www.owasp.org/index.php/Top_10_2013-A4-Insecure_Direct_Object_References

⁵ http://www.red-database-security.com/whitepaper/oracle_sql_injection_web.html

⁶ Java Null Byte Injections, December 2007, Arshan Dabirsiaghi <http://i8jesus.com/?p=9>

table of contents

search

Settings

queue

back

[6 Protecting Sensitive Data](#)

7 SQL Injection and Other Injection Attacks

[8 Safe File Upload and File I/O](#)

6h 33m remaining